Batch Write and Checksums

In the previous chapter, you already built a full LSM-based storage engine. At the end of this week, we will implement some easy but important optimizations of the storage engine. Welcome to Mini-LSM's week 2 snack time!

In this chapter, you will:

- Implement the batch write interface.
- Add checksums to the blocks, SST metadata, manifest, and WALs.

Note: We do not have unit tests for this chapter. As long as you pass all previous tests and ensure checksums are properly encoded in your file format, it would be fine.

Task 1: Write Batch Interface

In this task, we will prepare for week 3 of this course by adding a write batch API. You will need to modify:

```
src/lsm_storage.rs
```

The user provides `write_batch` with a batch of records to be written to the database. The records are `WriteBatchRecord<T: AsRef<[u8]>>`, and therefore it can be either `Bytes`, `&[u8]` or `Vec<u8>`. There are two types of records: delete and put. You may handle them in the same way as your `put` and `delete` function.

After that, you may refactor your original `put` and `delete` function to call `write_batch`.

You should pass all test cases in previous chapters after implementing this functionality.

Task 2: Block Checksum

In this task, you will need to add a block checksum at the end of each block when encoding the SST. You will need to modify:

```
src/table/builder.rs
src/table.rs
```

The format of the SST will be changed to:

```
-------------------------------------------------------------------------------
-----------------------------------------
|                   Block Section                     |
Meta Section                            |
-------------------------------------------------------------------------------
-----------------------------------------
| data block | checksum | ... | data block | checksum | metadata | meta block
offset | bloom filter | bloom filter offset |
|   varlen   |    u32   |     |   varlen   |    u32   |  varlen  |         u32
|    varlen    |        u32          |
-------------------------------------------------------------------------------
-----------------------------------------
```

We use crc32 as our checksum algorithm. You can use `crc32fast::hash` to generate the checksum for the block after building a block.

Usually, when user specify the target block size in the storage options, the size should include both block content and checksum. For example, if the target block size is 4096, and the checksum takes 4 bytes, the actual block content target size should be 4092. However, to avoid breaking previous test cases and for simplicity, in our course, we will **still** use the target block size as the target content size, and simply append the checksum at the end of the block.

When you read the block, you should verify the checksum in `read_block` correctly generate the slices for the block content. You should pass all test cases in previous chapters after implementing this functionality.

Task 3: SST Meta Checksum

In this task, you will need to add a block checksum for bloom filters and block metadata:

```
src/table.rs
src/table/bloom.rs
src/table/builder.rs
```

```
-------------------------------------------------------------------------------
--------------------------
|                                                 Meta Section
|
-------------------------------------------------------------------------------
--------------------------
| no. of block | metadata | checksum | meta block offset | bloom filter |
checksum | bloom filter offset |
|     u32      |  varlen  |    u32   |        u32        |    varlen    |
u32   |        u32          |
-------------------------------------------------------------------------------
--------------------------
```

You will need to add a checksum at the end of the bloom filter in `Bloom::encode` and `Bloom::decode`. Note that most of our APIs take an existing buffer that the implementation will write into, for example, `Bloom::encode`. Therefore, you should record the offset of the beginning of the bloom filter before writing the encoded content, and only checksum the bloom filter itself instead of the whole buffer.

After that, you can add a checksum at the end of block metadata. You might find it helpful to also add a length of metadata at the beginning of the section, so that it will be easier to know where to stop when decoding the block metadata.

Task 4: WAL Checksum

In this task, you will need to modify:

```
src/wal.rs
```

We will do a per-record checksum in the write-ahead log. To do this, you have two choices:

- Generate a buffer of the key-value record, and use `crc32fast::hash` to compute the checksum at once.
- Write one field at a time (e.g., key length, key slice), and use a `crc32fast::Hasher` to compute the checksum incrementally on each field.

This is up to your choice and you will need to *choose your own adventure*. Both method should produce exactly the same result, as long as you handle little endian / big endian correctly. The new WAL encoding should be like:

```
| key_len | key | value_len | value | checksum |
```

Task 5: Manifest Checksum

Lastly, let us add a checksum on the manifest file. Manifest is similar to a WAL, except that previously, we do not store the length of each record. To make the implementation easier, we now add a header of record length at the beginning of a record, and add a checksum at the end of the record.

The new manifest format is like:

```
| len | JSON record | checksum | len | JSON record | checksum | len | JSON
record | checksum |
```

After implementing everything, you should pass all previous test cases. We do not provide new test cases in this chapter.

Test Your Understanding

- Consider the case that an LSM storage engine only provides `write_batch` as the write interface (instead of single put + delete). Is it possible to implement it as follows: there is a single write thread with an mpsc channel receiver to get the changes, and all threads send write batches to the write thread. The write thread is the single point to write to the database. What are the pros/cons of this implementation? (Congrats if you do so you get BadgerDB!)
- Is it okay to put all block checksums altogether at the end of the SST file instead of store it along with the block? Why?

We do not provide reference answers to the questions, and feel free to discuss about them in the Discord community.

Bonus Tasks

- **Recovering when Corruption**. If there is a checksum error, open the database in a safe mode so that no writes can be performed and non-corrupted data can still be retrieved.